TechStore

HTML & CSS Code Documentation

E-Commerce Platform

March 2026

Document Version: 1.0

Technology Stack: React + TypeScript + Tailwind CSS

Table of Contents

1. Project Overview 3

2. Project Structure 4

3. Core Application Code 5

4. Component Architecture 6

5. Page Components 7

6. State Management 15

7. Styling System 16

8. Data Models 17

1. Project Overview

Technology Stack

- **React 18** - UI Framework
- **TypeScript** - Type-safe JavaScript
- **Tailwind CSS v4** - Utility-first CSS framework
- **React Router** - Client-side routing
- **Lucide React** - Icon library
- **Context API** - Global state management

Key Features

- 11 fully functional screens
- Shopping cart with persistent state
- Product catalog with categories
- User authentication flow
- Checkout process
- Responsive design for all screen sizes
- Modern, clean UI with consistent design system

2. Project Structure

```
/src
  /app
    App.tsx                  # Main application component
    /components
      Header.tsx             # Navigation header
      Footer.tsx             # Site footer
      Layout.tsx             # Page layout wrapper
      ProductCard.tsx        # Product display card
    /context
      CartContext.tsx        # Shopping cart state management
    /data
      products.ts            # Product catalog data
    /pages
      Home.tsx               # Homepage
      Shop.tsx               # Product listing
      ProductDetail.tsx      # Individual product page
      Cart.tsx               # Shopping cart
      Checkout.tsx           # Checkout form
      OrderSuccess.tsx       # Order confirmation
      Login.tsx              # User login
      Register.tsx           # User registration
      Account.tsx            # User dashboard
      About.tsx              # About page
      Contact.tsx            # Contact form
      Wireframes.tsx         # Wireframes documentation
      CodeDocumentation.tsx  # This code documentation
  /styles
    theme.css              # Global styles and CSS variables
    fonts.css              # Font imports
  main.tsx                 # Application entry point
  vite-env.d.ts           # TypeScript definitions
```

3. Core Application Code

App.tsx - Main Application

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import { CartProvider } from './context/CartContext';
import { Layout } from './components/Layout';
import { Home } from './pages/Home';
import { Shop } from './pages/Shop';
import { ProductDetail } from './pages/ProductDetail';
import { Cart } from './pages/Cart';
import { Checkout } from './pages/Checkout';
import { OrderSuccess } from './pages/OrderSuccess';
import { Login } from './pages/Login';
import { Register } from './pages/Register';
import { Account } from './pages/Account';
import { About } from './pages/About';
import { Contact } from './pages/Contact';
import { Wireframes } from './pages/Wireframes';

export default function App() {
  return (
    <Router>
      <CartProvider>
        <Routes>
          <Route path="/wireframes" element={<Wireframes />} />
          <Route
            path="*"
            element={
              <Layout>
                <Routes>
                  <Route path="/" element={<Home />} />
                  <Route path="/shop" element={<Shop />} />
                  <Route path="/product/:id" element={<ProductDetail />} />
                  <Route path="/cart" element={<Cart />} />
                  <Route path="/checkout" element={<Checkout />} />
                  <Route path="/order-success" element={<OrderSuccess />} />
                  <Route path="/login" element={<Login />} />
                  <Route path="/register" element={<Register />} />
                  <Route path="/account" element={<Account />} />
                  <Route path="/about" element={<About />} />
                  <Route path="/contact" element={<Contact />} />
                </Routes>
              </Layout>
            }
          />
        </Routes>
      </CartProvider>
    </Router>
  );
}
```

4. Component Architecture

Layout.tsx - Page Wrapper

```
import { ReactNode } from 'react';
import { Header } from './Header';
import { Footer } from './Footer';

interface LayoutProps {
  children: ReactNode;
}

export function Layout({ children }: LayoutProps) {
  return (
    <div className="flex flex-col min-h-screen">
      <Header />
      <main className="flex-1">
        {children}
      </main>
      <Footer />
    </div>
  );
}
```

Header.tsx - Navigation Component

```
import { Link } from 'react-router-dom';
import { ShoppingCart, User, Menu } from 'lucide-react';
import { useCart } from '../context/CartContext';
import { useState } from 'react';

export function Header() {
  const { items } = useCart();
  const [isMenuOpen, setIsMenuOpen] = useState(false);
  const itemCount = items.reduce((sum, item) => sum + item.quantity, 0);

  return (
    <header className="bg-slate-900 text-white sticky top-0 z-50">
      <div className="container mx-auto px-4">
        <div className="flex items-center justify-between h-16">
          <Link to="/" className="text-2xl font-bold">
            TechStore
          </Link>

          {/* Desktop Navigation */}
          <nav className="hidden md:flex items-center space-x-8">
            <Link to="/" className="hover:text-blue-400">Home</Link>
            <Link to="/shop" className="hover:text-blue-400">Shop</Link>
            <Link to="/about" className="hover:text-blue-400">About</Link>
            <Link to="/contact" className="hover:text-blue-400">Contact</Link>
          </nav>
```

```
          {/* Icons */}
          <div className="flex items-center space-x-4">
            <Link to="/account" className="hover:text-blue-400">
              <User className="w-6 h-6" />
            </Link>
            <Link to="/cart" className="relative hover:text-blue-400">
              <ShoppingCart className="w-6 h-6" />
              {itemCount > 0 && (
                <span className="absolute -top-2 -right-2 bg-blue-600 text-white
                  rounded-full w-5 h-5 flex items-center justify-center text-xs">
                  {itemCount}
                </span>
              )}
            </Link>
            <button
              className="md:hidden"
              onClick={() => setIsMenuOpen(!isMenuOpen)}
            >
              <Menu className="w-6 h-6" />
            </button>
          </div>
        </div>

        {/* Mobile Menu */}
        {isMenuOpen && (
          <nav className="md:hidden py-4 space-y-2">
            <Link to="/" className="block py-2 hover:text-blue-400">Home</Link>
            <Link to="/shop" className="block py-2 hover:text-blue-400">Shop</Link>
            <Link to="/about" className="block py-2 hover:text-blue-400">About</Link>
            <Link to="/contact" className="block py-2 hover:text-blue-400">Contact</Link>
          </nav>
        )}
      </div>
    </header>
  );
}
```

5. Page Components

Home.tsx - Homepage Component

The homepage features a hero section, featured products, and category highlights.

```
import { Link } from 'react-router-dom';
import { products } from '../data/products';
import { ProductCard } from '../components/ProductCard';

export function Home() {
  const featuredProducts = products.slice(0, 4);

  return (
    <div>
      {/* Hero Section */}
      <section className="bg-gradient-to-r from-blue-600 to-purple-600
        text-white py-20">
        <div className="container mx-auto px-4 text-center">
          <h1 className="text-5xl md:text-6xl mb-6">
            Welcome to TechStore
          </h1>
          <p className="text-xl md:text-2xl mb-8">
            Discover the latest in tech and electronics
          </p>
          <Link to="/shop"
            className="bg-white text-blue-600 px-8 py-3 rounded-lg text-lg
              font-semibold hover:bg-slate-100 inline-block">
            Shop Now
          </Link>
        </div>
      </section>

      {/* Featured Products */}
      <section className="py-16">
        <div className="container mx-auto px-4">
          <h2 className="text-3xl md:text-4xl mb-8 text-center">
            Featured Products
          </h2>
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4
            gap-6">
            {featuredProducts.map(product => (
              <ProductCard key={product.id} product={product} />
            ))}
          </div>
        </div>
      </section>
    </div>
  );
}
```

Shop.tsx - Product Listing

```
import { useState } from 'react';
import { products } from '../data/products';
import { ProductCard } from '../components/ProductCard';

export function Shop() {
  const [selectedCategory, setSelectedCategory] = useState('all');

  const categories = ['all', 'electronics', 'accessories', 'gadgets'];

  const filteredProducts = selectedCategory === 'all'
    ? products
    : products.filter(p => p.category === selectedCategory);

  return (
    <div className="container mx-auto px-4 py-8">
      <h1 className="text-4xl mb-8">Shop</h1>

      {/* Category Filter */}
      <div className="mb-8 flex flex-wrap gap-4">
        {categories.map(category => (
          <button
            key={category}
            onClick={() => setSelectedCategory(category)}
            className={`px-6 py-2 rounded-lg capitalize
              ${selectedCategory === category
                ? 'bg-blue-600 text-white'
                : 'bg-slate-200 text-slate-800 hover:bg-slate-300'
              }`}
          >
            {category}
          </button>
        ))}
      </div>

      {/* Product Grid */}
      <div className="grid grid-cols-1 md:grid-cols-3 lg:grid-cols-4 gap-6">
        {filteredProducts.map(product => (
          <ProductCard key={product.id} product={product} />
        ))}
      </div>
    </div>
  );
}
```

ProductDetail.tsx - Product Page

```
import { useParams, Link } from 'react-router-dom';
import { products } from '../data/products';
import { useCart } from '../context/CartContext';
import { ShoppingCart, ArrowLeft } from 'lucide-react';
import { useState } from 'react';

export function ProductDetail() {
  const { id } = useParams();
  const { addItem } = useCart();
  const [quantity, setQuantity] = useState(1);

  const product = products.find(p => p.id === id);

  if (!product) {
    return <div className="container mx-auto px-4 py-8">
      Product not found
    </div>;
  }

  const handleAddToCart = () => {
    addItem(product, quantity);
  };

  return (
    <div className="container mx-auto px-4 py-8">
      <Link to="/shop" className="flex items-center text-blue-600
        hover:text-blue-700 mb-6">
        <ArrowLeft className="w-4 h-4 mr-2" />
        Back to Shop
      </Link>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
        <div className="aspect-square bg-slate-100 rounded-lg
          overflow-hidden">
          <img
            src={product.image}
            alt={product.name}
            className="w-full h-full object-cover"
          />
        </div>

        <div>
          <h1 className="text-4xl mb-4">{product.name}</h1>
          <p className="text-3xl text-blue-600 mb-6">
            ${product.price}
          </p>
          <p className="text-lg text-slate-600 mb-8">
            {product.description}
          </p>

          <div className="flex items-center gap-4 mb-6">
            <label className="text-lg">Quantity:</label>
            <input
              type="number"
```

```
              min="1"
              value={quantity}
              onChange={(e) => setQuantity(parseInt(e.target.value))}
              className="border border-slate-300 rounded px-4 py-2 w-20"
            />
          </div>

          <button
            onClick={handleAddToCart}
            className="bg-blue-600 text-white px-8 py-3 rounded-lg
              hover:bg-blue-700 flex items-center gap-2"
          >
            <ShoppingCart className="w-5 h-5" />
            Add to Cart
          </button>
        </div>
      </div>
    </div>
  );
}
```

Cart.tsx - Shopping Cart

```
import { Link } from 'react-router-dom';
import { useCart } from '../context/CartContext';
import { Trash2, Plus, Minus } from 'lucide-react';

export function Cart() {
  const { items, removeItem, updateQuantity, total } = useCart();

  if (items.length === 0) {
    return (
      <div className="container mx-auto px-4 py-16 text-center">
        <h1 className="text-4xl mb-4">Your Cart is Empty</h1>
        <Link to="/shop" className="text-blue-600 hover:text-blue-700">
          Continue Shopping
        </Link>
      </div>
    );
  }

  return (
    <div className="container mx-auto px-4 py-8">
      <h1 className="text-4xl mb-8">Shopping Cart</h1>

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
        <div className="lg:col-span-2 space-y-4">
          {items.map(item => (
            <div key={item.product.id}
              className="flex gap-4 bg-white p-4 rounded-lg border">
              <img
                src={item.product.image}
                alt={item.product.name}
                className="w-24 h-24 object-cover rounded"
              />
```

```
              <div className="flex-1">
                <h3 className="text-xl mb-2">{item.product.name}</h3>
                <p className="text-blue-600">${item.product.price}</p>
              </div>
              <div className="flex items-center gap-2">
                <button onClick={() =>
                  updateQuantity(item.product.id, item.quantity - 1)}
                  className="p-1 hover:bg-slate-100 rounded"
                >
                  <Minus className="w-4 h-4" />
                </button>
                <span className="w-8 text-center">{item.quantity}</span>
                <button onClick={() =>
                  updateQuantity(item.product.id, item.quantity + 1)}
                  className="p-1 hover:bg-slate-100 rounded"
                >
                  <Plus className="w-4 h-4" />
                </button>
                <button onClick={() => removeItem(item.product.id)}
                  className="ml-4 text-red-600 hover:text-red-700"
                >
                  <Trash2 className="w-5 h-5" />
                </button>
              </div>
            </div>
          ))}
        </div>

        <div className="bg-slate-100 p-6 rounded-lg h-fit">
          <h2 className="text-2xl mb-4">Order Summary</h2>
          <div className="space-y-2 mb-4">
            <div className="flex justify-between">
              <span>Subtotal</span>
              <span>${total.toFixed(2)}</span>
            </div>
            <div className="flex justify-between">
              <span>Shipping</span>
              <span>Free</span>
            </div>
          </div>
          <div className="border-t pt-4 mb-4">
            <div className="flex justify-between text-xl font-bold">
              <span>Total</span>
              <span>${total.toFixed(2)}</span>
            </div>
          </div>
          <Link to="/checkout"
            className="block w-full bg-blue-600 text-white text-center
              py-3 rounded-lg hover:bg-blue-700">
            Proceed to Checkout
          </Link>
        </div>
      </div>
    </div>
  );
}
```

6. State Management

CartContext.tsx - Shopping Cart State

```
import { createContext, useContext, useState, ReactNode } from 'react';
import { Product } from '../data/products';

interface CartItem {
  product: Product;
  quantity: number;
}

interface CartContextType {
  items: CartItem[];
  addItem: (product: Product, quantity?: number) => void;
  removeItem: (productId: string) => void;
  updateQuantity: (productId: string, quantity: number) => void;
  clearCart: () => void;
  total: number;
}

const CartContext = createContext<CartContextType | undefined>(undefined);

export function CartProvider({ children }: { children: ReactNode }) {
  const [items, setItems] = useState<CartItem[]>([]);

  const addItem = (product: Product, quantity = 1) => {
    setItems(current => {
      const existing = current.find(item =>
        item.product.id === product.id);
      if (existing) {
        return current.map(item =>
          item.product.id === product.id
            ? { ...item, quantity: item.quantity + quantity }
            : item
        );
      }
      return [...current, { product, quantity }];
    });
  };

  const removeItem = (productId: string) => {
    setItems(current => current.filter(item =>
      item.product.id !== productId));
  };

  const updateQuantity = (productId: string, quantity: number) => {
    if (quantity < 1) {
      removeItem(productId);
      return;
    }
    setItems(current => current.map(item =>
      item.product.id === productId
        ? { ...item, quantity }
```

```
        : item
    ));
  };

  const clearCart = () => setItems([]);

  const total = items.reduce((sum, item) =>
    sum + item.product.price * item.quantity, 0);

  return (
    <CartContext.Provider value={{
      items,
      addItem,
      removeItem,
      updateQuantity,
      clearCart,
      total
    }}>
      {children}
    </CartContext.Provider>
  );
}

export function useCart() {
  const context = useContext(CartContext);
  if (!context) {
    throw new Error('useCart must be used within CartProvider');
  }
  return context;
}
```

7. Styling System

Tailwind CSS Configuration

The application uses Tailwind CSS v4 with custom theme variables defined in theme.css.

theme.css - Global Styles

```
@import "tailwindcss";

@theme {
  --color-slate-50: #f8fafc;
  --color-slate-100: #f1f5f9;
  --color-slate-200: #e2e8f0;
  --color-slate-300: #cbd5e1;
  --color-slate-600: #475569;
  --color-slate-700: #334155;
  --color-slate-800: #1e293b;
  --color-slate-900: #0f172a;

  --color-blue-400: #60a5fa;
  --color-blue-600: #2563eb;
  --color-blue-700: #1d4ed8;

  --color-purple-600: #9333ea;

  --color-red-600: #dc2626;
  --color-red-700: #b91c1c;

  --color-green-600: #16a34a;
}

body {
  font-family: system-ui, -apple-system, sans-serif;
  background-color: #f8fafc;
  color: #0f172a;
}
```

Common Tailwind Patterns

Container: `container mx-auto px-4`

Button Primary: `bg-blue-600 text-white px-8 py-3 rounded-lg hover:bg-blue-700`

Card: `bg-white p-6 rounded-lg shadow-md`

Grid Layout: `grid grid-cols-1 md:grid-cols-3 gap-6`

8. Data Models

products.ts - Product Data

```
export interface Product {
  id: string;
  name: string;
  price: number;
  category: string;
  image: string;
  description: string;
}

export const products: Product[] = [
  {
    id: '1',
    name: 'Wireless Headphones',
    price: 99.99,
    category: 'electronics',
    image: 'https://images.unsplash.com/photo-1505740420928-5e560c06d30e',
    description: 'Premium wireless headphones with noise cancellation'
  },
  {
    id: '2',
    name: 'Smart Watch',
    price: 299.99,
    category: 'electronics',
    image: 'https://images.unsplash.com/photo-1523275335684-37898b6baf30',
    description: 'Feature-rich smartwatch with health tracking'
  },
  // ... more products
];
```

TypeScript Interfaces

```
// Product Interface
interface Product {
  id: string;
  name: string;
  price: number;
  category: string;
  image: string;
  description: string;
}

// Cart Item Interface
interface CartItem {
  product: Product;
  quantity: number;
}
```

```
// User Interface (for authentication)
interface User {
  id: string;
  email: string;
  name: string;
}

// Order Interface
interface Order {
  id: string;
  items: CartItem[];
  total: number;
  date: string;
  status: string;
}
```

9. Common CSS Patterns

Responsive Design

```
/* Mobile First Approach */
<div className="
  grid
  grid-cols-1          /* 1 column on mobile */
  md:grid-cols-2       /* 2 columns on tablet */
  lg:grid-cols-4       /* 4 columns on desktop */
  gap-6
">

/* Text Sizing */
<h1 className="
  text-4xl             /* Base size */
  md:text-5xl          /* Tablet */
  lg:text-6xl          /* Desktop */
">

/* Padding/Margin */
<section className="
  py-8                 /* Mobile: 2rem vertical padding */
  md:py-12             /* Tablet: 3rem */
  lg:py-16             /* Desktop: 4rem */
">
```

Component Styling Examples

```
/* Card Component */
.product-card {
  background: white;
  border-radius: 0.5rem;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
  padding: 1.5rem;
  transition: transform 0.2s;
}

.product-card:hover {
  transform: translateY(-4px);
  box-shadow: 0 10px 15px rgba(0,0,0,0.1);
}

/* Button Styles */
.btn-primary {
  background-color: #2563eb;
  color: white;
  padding: 0.75rem 2rem;
  border-radius: 0.5rem;
  font-weight: 600;
  transition: background-color 0.2s;
```

```
}

.btn-primary:hover {
  background-color: #1d4ed8;
}

/* Input Fields */
.input-field {
  border: 1px solid #cbd5e1;
  border-radius: 0.5rem;
  padding: 0.75rem 1rem;
  width: 100%;
  transition: border-color 0.2s;
}

.input-field:focus {
  outline: none;
  border-color: #2563eb;
  box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.1);
}
```

10. Routing & Navigation

Route Structure

`/`	Home page
`/shop`	Product listing
`/product/:id`	Product detail page
`/cart`	Shopping cart
`/checkout`	Checkout form
`/order-success`	Order confirmation
`/login`	User login
`/register`	User registration
`/account`	User dashboard
`/about`	About page
`/contact`	Contact form

11. Best Practices & Guidelines

Component Structure

- Keep components small and focused on a single responsibility
- Use TypeScript interfaces for type safety
- Extract reusable logic into custom hooks
- Use React Context for global state (e.g., cart, auth)

Styling Guidelines

- Use Tailwind utility classes for styling
- Follow mobile-first responsive design
- Maintain consistent spacing and sizing
- Use CSS variables for theme colors

Performance

- Lazy load images with proper alt text
- Optimize images for web (WebP format when possible)
- Use React.memo for expensive components
- Minimize re-renders with proper dependency arrays

Accessibility

- Use semantic HTML elements
- Provide proper ARIA labels
- Ensure keyboard navigation works
- Maintain sufficient color contrast